

Don't Go Broke

Before the Agent Starts

Where This Booklet Fits:

This is booklet 3 of 5 in the Don't Go Broke series.

Disclaimer: This booklet provides general educational information and is not formal financial, legal, or professional engineering advice.

Table of Contents

- Field Note Opening
- 1. The Agent Is Not Just Typing Suggestions
- 2. Decide What the Agent May Change
- 3. Decide What the Agent Must Not Touch
- 4. Define the Safe First Task
- 5. Set Tool and Command Boundaries
- 6. Decide What the Agent May Not Decide Alone
- 7. Make the Agent Stop and Explain
- 8. Review Before the Next Step
- 9. Boundary Checklist
- Field Note Closing

Before the Agent Starts

Field Note Opening

You have your blueprint. You know what you want to build. Now, you are staring at an empty chat box connected to an AI coding agent.

Your instinct is to type: *"Build the dashboard."*

Stop.

The moment before you hit "Enter" on a broad task is the most dangerous moment in your build.

If you are using an autonomous coding agent, you are no longer just receiving code suggestions. You have handed the keyboard to an incredibly fast, highly capable junior developer who never sleeps, never hesitates, and—crucially—never asks for permission unless you force it to.

Before the agent starts acting, you must define its boundaries. You must tell it what it is allowed to touch, what tools it is allowed to use, and exactly when it must stop and wait for your approval.

Speed without direction isn't progress. It's just faster chaos. This guide is about putting up the guardrails before you step on the gas.

1. The Agent Is Not Just Typing Suggestions

If you have used ChatGPT to write a Python script, or if you use GitHub Copilot to autocomplete a function, you are used to passive AI.

Passive AI types where your cursor is. It waits for you to ask a question. It gives you a block of code, and you decide where to paste it.

Agentic coding tools are different. They are autonomous.

An agent can read your entire codebase. It can open files, delete lines of code, and create new directories. It can run commands in your terminal. If it encounters an error, it doesn't wait for you to fix it; it will try to fix the error itself.

The preparation required for a passive tool is completely different from the management required for an active agent. You are not managing a spellchecker. You are managing a worker. And if you give a fast worker a vague instruction, they will guess what you meant.

When an agent guesses, it usually guesses wrong. It will over-engineer simple solutions, install heavy libraries you don't need,

and rewrite working code because it thought it was "cleaning up."

2. Decide What the Agent May Change

Because an agent can touch anything in your project, your first job is to tell it exactly what it is allowed to touch today.

If you ask a human contractor to paint your kitchen, you don't want to come home and find they also demolished your bathroom because they thought the tile looked outdated. Agents will do exactly this to your code.

You must define the allowed scope in plain language.

Vague: *"Style the login button."* **Bounded:** *"You may only read `src/components/Login.js` and edit `src/css/styles.css` to style the login button."*

Restrict the agent's playing field to only what is strictly necessary for the immediate task. If you are building a new button, the agent has no business reading your database schema or editing your routing logic.

3. Decide What the Agent Must Not Touch

Defining what the agent *can* touch is only half the battle. You must explicitly protect your core assets.

Every project has critical load-bearing code. It might be your authentication logic, your payment processing integration, or a complicated database query that finally works after three days of debugging.

If you don't understand how a piece of working code works yet, the agent should not be allowed to rewrite it.

Introduce "do not touch" boundaries early in your prompt.

Example Boundary: *"Do not modify the `auth/` directory under any circumstances. If a feature requires changing how authentication works, stop and ask me first."*

Agents love to "clean up" code. They will see a working configuration file, decide it isn't using the latest syntax, and rewrite it. Suddenly, your app won't deploy, and you have no idea why. Protect your working parts.

4. Define the Safe First Task

First-time builders often treat agents like magic wands. They drop massive, multi-step requests into the prompt window.

- “Clean this up.”
- “Fix everything.”
- “Make it better.”
- “Finish the app.”
- “Add auth and deploy it.”

These are not tasks; they are wishes. And when an agent is given a wish, it will try to fulfill it all at once, opening a dozen files and tangling the logic into a knot.

You must break your blueprint down into a **Safe First Task**.

A safe first task is narrow, isolated, and highly verifiable. It has a clear definition of "done."

Unsafe: *"Build the user dashboard."* (The agent will guess the layout, guess the data structure, install charting libraries, and write fake API endpoints.)

Safe: *"Create the empty HTML and CSS layout container for the user dashboard. Do not connect it to data yet. Just build the visual shell."*

Start with a small, testable chunk of work. Verify it works. Then give the agent the next small piece.

5. Set Tool and Command Boundaries

Agents are increasingly capable of running commands directly in your terminal. This is a massive productivity boost, but it is also the fastest way to break your computer.

An agent can run `npm install` to add new dependencies. It can run database migrations. It can push code to production.

You must set hard boundaries around what commands the agent is allowed to run without your explicit, manual approval.

Example Boundary: *"You may run local development servers (like `npm run dev`) and test suites. You may NOT run any `npm install` commands, database migrations, or deployment scripts without asking me first."*

If the agent needs a new third-party library to solve a problem, it should ask you. You do not want to wake up to find your simple website now requires a heavy enterprise state-management library just because the agent thought it was convenient.

6. Decide What the Agent May Not Decide Alone

Agents will make silent architectural decisions unless you stop them.

If you tell an agent to "save user preferences," it has to decide *how* to save them. Should it use local storage? A database? A flat file?

If you haven't told it how, it will choose for you. It might choose a complex, expensive database solution when all you needed was a simple text file.

Explicitly list the decisions the agent may not make silently:

- Architecture changes
- Dependency choices (adding new libraries)
- Database schema changes
- Restructuring or moving files
- Changing authentication or payment flows

If the agent hits a fork in the road, its default behavior shouldn't be to pick a path. Its default behavior should be to stop and ask the human.

7. Make the Agent Stop and Explain

You dictate the workflow rhythm. The agent stops when you say stop.

The most effective boundary you can set is an approval gate. Before the agent writes a single line of code, require it to explain its plan.

Example Instruction: *"Before you edit any files, write a short summary of exactly what you plan to change and which files you plan to touch. Wait for my approval before proceeding."*

It is infinitely easier to correct a bad plan written in plain English than it is to untangle bad code spread across five different files.

When the agent finishes a task, require it to summarize what it actually did. *"I have finished the layout. I edited `index.html` and `styles.css`. Please review the visual layout in your browser before we proceed to adding data."*

8. Review Before the Next Step

Scope creep is the enemy of the first-time builder.

You ask the agent to fix the padding on a button. It fixes the padding, notices a typo in the header, fixes the typo, notices the routing logic is outdated, updates the router, breaks the app, and spends the next hour trying to fix the router.

To defend against scope creep, you must enforce the review gate.

When the agent finishes its Safe First Task, you must review the output before giving it the next instruction. You must verify that the button looks right *before* you let it start writing database queries.

This sets up the most critical phase of building with AI: trusting the code. Setting boundaries keeps the agent from running wild, but once the code is written, you have to verify it actually works. We will cover exactly how to do that in the next guide: *Before You Trust the First Draft*.

9. Boundary Checklist

Before you hit "Enter" on your first agent prompt, fill out this checklist. You can paste this directly into your agent's instructions, or keep it on your desk as a mental check.

- **Agent Task:** [Exactly what is the narrow, verifiable task?]
- **Allowed Files/Folders:** [Which specific files is the agent allowed to edit?]
- **Off-Limits Files/Folders:** [Which critical files must the agent never touch?]
- **Allowed Tools/Commands:** [What terminal commands can the agent run freely?]
- **Commands Requiring Approval:** [What must the agent ask before running? e.g., installs, deployments]
- **Decisions Requiring Approval:** [What architectural or dependency choices must the agent ask about?]
- **Stop Point:** [At what exact moment should the agent stop working?]
- **Review Requirement:** [What must the agent show you before it proceeds?]
- **Definition of Done:** [How will we know this specific task is complete?]
- **If Unsure:** [If you encounter ambiguity, do not guess. Stop and ask me for clarification.]

Field Note Closing

The agent should not start until the boundaries are clear.

Taking five minutes to define what the agent cannot do will save you five hours of trying to figure out what it just did.

The goal of setting these boundaries isn't to slow you down. The goal is to keep you in the driver's seat. You are the builder. The agent is just the hands on the keyboard.

Keep the tasks small. Keep the boundaries tight. When you're ready, give the agent its first instruction.

The Next Step

You are ready for the next phase. Proceed to booklet: **Before You Trust the First Draft.**